

Numpy

This document contains useful information on how to use the Python numpy module.

Note: The terminology included in this document and knowledge of numpy is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction	2
Terminology	2
Getting Started	2
Array Fundamentals	3
Creation	3
Indexing	5
Slicing	6
Vectorized Operations	6
Masking	7
Bitwise Operators	7
Handling Missing Data	8
Array Properties	9
Attributes	9
Methods	10
Applying Functions and Methods	11
Aggregate Functions and Methods	11
Numpy Functions	12
Generating Random Numbers	15
Matrix Operations	15
Matrix Multiplication	15
Matrix Inversion and the Determinant	16
Eigenvalues and Eigenvectors	16
Resources	16

Introduction

The numpy module adds support for large multi-dimensional arrays and matrices, along with a large collection of high-level [mathematical functions](#) to operate on these arrays. Numpy is written partially in Python, but most of the parts that require fast computation are written in C or C++. Numpy is used frequently in data manipulation, machine learning, image processing, and many others. In order to effectively introduce you to numpy, we need to establish common ground regarding the terminology and software we will be using in this course.

Terminology

Numpy - an extension to Python that uses multi-dimensional arrays alongside high-level mathematical functions that can be used on them

Array - an n-dimensional, fixed-size object that holds homogeneous data types

Masking - creating an array of True/False values and using it to only include values of the original array that fulfill the conditional requirement

Vectorized Operations - using built-in operators on arrays to run mathematical operators faster with much simpler and easy-to-understand syntax

NaN - is an acronym for Not a Number and is a NumPy data type used as a placeholder for missing values

Getting Started

By convention, the numpy module is imported with the following line of code; `import numpy as np`. Numpy should have been installed with Anaconda or Miniconda, however if the import throws an error, execute `conda install numpy` from the command line.

Array Fundamentals

The core of the numpy module is the array object, a multidimensional, homogeneous array of fixed length. In comparison to lists, they take up significantly less space, are faster when reading or writing, and are used to compute high-level mathematical operations more efficiently than python lists.

Creation

There are many ways to create an array, but we will demonstrate the seven used in this class:

1. `np.array(object, dtype=None, ndmin=0)`

where `object` is any array-like, nested sequence (typically lists or tuples), `dtype` is the type of the nested elements of the sequence (see [dtypes](#) for common dtypes), `ndmin` specifies the minimum number of dimensions the array has (think 1D, 2D, etc.).

This method creates an array object of the `object` parameter, defaulting to its `dtype`.

```
>>> np.array([1,2,3,4,5], dtype="int32", ndmin=1)
array([1,2,3,4,5])
```

2. `np.zeros(shape, dtype=float)`

where `shape` is the shape of the array (int or tuple of ints in the form (row, column)) and `dtype` is the data type of the nested elements.

This method creates an array object of all zeros, defaulting to the `dtype` of float64.

```
>>> np.zeros((2,3))
array([[0., 0., 0.],
       [0., 0., 0.]])
```

Reminder: when declaring an array's shape, remember to use a tuple or list!

3. `np.ones(shape, dtype=float)`

where `shape` is the shape of the array and `dtype` is the data type of the nested elements.

This method creates an array object of all ones, defaulting to the `dtype` of float64.

```
>>> np.ones(3, dtype="int32")
array([1, 1, 1])
```

4. `np.full(shape, fill_value, dtype=None)`

where `shape` is the shape of the array, `fill_value` is a scalar you want to fill the array with, and `dtype` is the data type of the nested elements.

This method creates an array object of all the specified value, defaulting to the dtype of the value.

```
>>> np.full((3,3), 8)
array([[8, 8, 8],
       [8, 8, 8],
       [8, 8, 8]])
```

5. `np.arange(start, stop, step, dtype=None)`

where `start` is the starting number (**inclusive**), `stop` is the ending number (**exclusive**), and `step` is the step (the absolute value of the difference of any two consecutive numbers). One thing to note is that `np.arange` can take step sizes that are not a whole number while `range` can only take whole number step sizes.

This method creates an array object of a range of numbers using `start`, `stop`, `step`, defaulting to its dtype.

```
>>> np.arange(4, 16, 3)
array([4, 7, 10, 13])
>>> np.arange(1, 3.5, .5)
array([1. , 1.5, 2. , 2.5, 3. ])
```

6. `np.linspace(start, stop, num=50, endpoint=True, dtype=float)`

where `start` is the starting number (**inclusive**), `stop` is the ending number (**inclusive**), `num` is the number of samples to generate, `endpoint` is the boolean value of whether or not `stop` is inclusive, and `dtype` is the data type of nested elements.

This method creates an array object of a range of evenly-spaced numbers with a total of `num` numbers, including **both** the start and stop, and defaults to float64.

```
>>> np.linspace(0, 10, 5)
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

7. `np.random.random((rows, columns))`

where `rows` is the number of rows and `columns` is the number of columns.

This method creates an array of random uniform distribution floats in range `[0, 1)`.

```
>>> np.random.random((1,2))  
array([[0.70612481, 0.07211607]])
```

8. `np.random.randint(low, high = None, size = None, dtype = int)`

where `low` is the starting number (inclusive) and `high` is the ending number (exclusive).

This method creates an array of random uniform distribution integers in range `[low, high)`.

```
>>> np.random.randint(3,9)  
6  
  
>>> np.random.randint(3,9,size=4)  
array([4, 6, 6, 5])
```

Indexing

Like with lists, arrays can be indexed into using subscription (with square brackets), and values used to index into the array can be positive or negative.

If the array is one-dimensional (only contains one row), singular square brackets will index into the array much like a list.

```
>>> arr = np.array([1,2,3,4,5])  
  
>>> arr[-3]  
3
```

If the array is two-dimensional (and up), the surface of interest is distinguished by the order of the indices and separated by a comma. For 2-dimensional arrays, the order is `[rows, columns]`.

```
>>> arr = np.array([[1, 2, 3],[4, 5, 6]])  
  
>>> arr[1,2]  
6  
  
>>> arr[1]
```

```
[4, 5, 6]
```

Slicing

Arrays can be sliced like lists and follow the order of [rows, columns], each of which can be sliced individually.

```
>>> arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> arr[1:,1:]
array([[5, 6],
       [8, 9]])
>>> arr[:,2:]
array([[3],
       [6],
       [9]])
```

Vectorized Operations

Vectorized operations use concise syntax to apply mathematical operations or boolean expressions element wise to an array. Consider the following examples.

```
>>> arr = np.array([1, 2, 3])
>>> arr * 2
array([2, 4, 6])
>>> arr - 2
array([-1, 0, 1])
>>> arr + arr
array([2, 4, 6])
>>> arr <= 2
array([True, True, False])
```

Notice that all of the examples above **did not** change the original array but instead returned a new array. This is a very important property that will continuously appear as you gain experience using numpy.

Masking

Sometimes it is desirable to filter an array to only include a subset of the original. Masking uses a boolean expression to create an array of booleans that is then used to index into the original array.

Step 1: Create an Array

```
>>> arr = np.arange(0, 20, 3)
```

Step 2: Create the Boolean Array with the Expression

```
>>> bool_arr = arr % 2 == 0
```

Step 3: Index into the Original Array

```
>>> arr[bool_arr]
array([0,  6, 12, 18])
```

You may also skip Step 2 and use the boolean expression directly inside the square brackets of the original array.

```
>>> arr[arr % 2 == 0]
array([0,  6, 12, 18])
```

Bitwise Operators

Oftentimes it is desirable to apply boolean operators to numpy arrays. Due to the nature of arrays we cannot use normal logical operators like *not*, *and*, *or* but instead we should use bitwise operators `~`, `&`, `|` accordingly. Consider the following example.

Suppose we have an array of integers

```
>>> arr = np.arange(1, 23, 3)
>>> arr
array([ 1,  4,  7, 10, 13, 16, 19, 22])
```

and we want to count the number of values that are even and less than 13. Let's break this apart into 3 steps.

First we'll create a boolean array where True represents even and False represents odd.

```
>>> arr % 2 == 0
array([False,  True, False,  True, False,  True, False,  True])
```

Secondly we'll create a boolean array where True represents less than 13 and False represents 13 or greater.

```
>>> arr < 13  
array([ True,  True,  True,  True, False, False, False, False])
```

Lastly, we'll use the bitwise and operator, &, to combine both expressions and then take the sum. Note that parenthesis must be used to separate each condition and the bitwise operator.

```
>>> np.sum((arr % 2 == 0) & (arr < 13))  
  
2
```

Now that we know there are 2 values that are even and less than 13, how can we retrieve those values from the array? In other words, how can we *filter* the original array to only include values that are even and less than 13? To accomplish this we can mask the original array with the boolean array created previously.

```
>>> arr[(arr % 2 == 0) & (arr < 13)]  
  
array([4, 10])
```

Handling Missing Data

Oftentimes when reading data there are missing values; knowing how to manage the missing values is very important. Rather than using None (which in Python is of the data type NoneType), NumPy provides a floating point constant to represent a missing numeric value. NaN stands for Not a Number. Because NaN is not the same thing as None or null, there are some interesting things about the way it should be used in our code.

The first fundamental thing to note about np.nan is that np.nan does not *equal* np.nan, instead np.nan *is* np.nan.

```
>>> np.nan == np.nan  
  
False  
  
>>> np.nan is np.nan  
  
True
```

Due to the fact that numpy arrays must be of homogeneous data types, sometimes using the pure python *is* operator yields unintuitive results.

```
>>> arr = np.array([-2, 1.5, np.nan, 2, -5], dtype = float)  
  
>>> arr[2] is np.nan
```

False

In this example, the `np.nan` within the numpy array was converted to type `numpy.float64` so that vectorized operations can be performed despite the array holding nan values.

```
>>> arr * 2
array([ -4.,   3.,  nan,   4., -10.] )
```

After this conversion, we are comparing two nan values of different types which yields undesirable results.

Now to check if a value is nan, we can either have an array of dtype object (which is not recommended because numpys performance decreases substantially) or we can use numpys built in `isnan` function, which is most common.

```
>>> arr = np.array([-2, 1.5, np.nan, 2, -5], dtype = object)
>>> arr[2] is np.nan
True
```

```
>>> arr = np.array([-2, 1.5, np.nan, 2, -5], dtype = float)
>>> np.isnan(arr[2])
True
```

It's worth noting that the `np.isnan` function is a vectorized function therefore it can be applied elementwise to filter out `np.nan` values.

```
>>> arr = np.array([-2, 1.5, np.nan, 2, -5], dtype = float)
>>> np.isnan(arr)
array([False, False,  True, False, False])
>>> arr[~np.isnan(arr)]
array([-2.,  1.5,  2., -5.] )
```

Finally, something to be careful of is that the `np.isnan` function does not work on an array with dtype object. In cases like this, convert the arrays data type to object and use the python `is` operator or if possible convert the arrays data type to float and use `np.isnan`.

Array Properties

Attributes

Array attributes hold information about the array. Here is a list of commonly used attributes.

.dtype - returns the data type of the elements within an array
.ndim - returns the number of dimensions of an array
.size - returns the number of elements in an array
.shape - returns the shape in the order (rows, columns) of an array

```
>>> arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
>>> arr.shape  
(3, 3)
```

.T - Returns the transpose of the original array

```
>>> arr.T  
array([[1, 4, 7],  
       [2, 5, 8],  
       [3, 6, 9]])
```

Methods

Array methods allow the user to return a new array or modify an existing array. Here are some useful methods:

.copy() - returns a copy of an array that can be assigned to another variable. Copy is used when two identical independent arrays are wanted.

.fill(value) - replaces all elements of an array with the specified value but does not return it

.reshape(rows, columns) - returns an array with the new shape but does not change it

```
>>> arr = np.array([1, 2, 3, 4, 5, 6])  
>>> arr.reshape((2,3))  
array([[1, 2, 3],  
       [4, 5, 6]])
```

.resize(rows, columns) - changes the shape of an array but does not return the array

```
>>> arr = np.array([1, 2, 3, 4, 5, 6])  
>>> arr.resize((2,3))  
>>> arr  
array([[1, 2, 3],  
       [4, 5, 6]])
```

.flatten() - Returns the array, “flattened” to condense all elements into a one-dimensional array

```
>>> arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> arr.flatten()
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

.sort(axis = -1) - Sorts the array in ascending order, defaulting to the last axis

```
>>> arr = np.array([6, 1, 4, 2, 5, 3])
>>> arr.sort()
>>> arr
array([1, 2, 3, 4, 5, 6])
```

Note that `.sort()` has a function counterpart called `np.sort()` that accomplishes the same function and has the same attributes but returns the sorted array instead of sorting it in place.

.transpose() - Returns the transpose of the original array (same function as the `.T` attribute)

.astype(dtype) - Returns the array, with all values cast to a specified dtype

Applying Functions and Methods

Aggregate Functions and Methods

The following aggregates can be applied as both a function, i.e. `array.max()`, and a method, i.e. `np.max(array)`.

.max or np.max - returns the maximum of all the elements of an array or a given axis

.mean or np.mean - returns the mean of all the elements of an array or a given axis

.min or np.min - returns the minimum of all the elements of an array or a given axis

.sum or np.sum - returns the sum of all the elements of an array or a given axis

The axis parameter allows us to apply a function across a given dimension. Consider the following example for an illustration of this concept.

Suppose we have an array that holds exam grades. We will first define the array then slice the data to only include the grades.

```
>>> arr = np.array(["Exam 1", "Exam 2", "Exam 3"],
                    [          90,          87,          96],
```

```

[      79,      100,      92],
[      98,      60,      74]])

```

```
>>> grades = np.array(arr[1:, :], dtype = "int32")
```

Now if we wanted to find the maximum grade across Exam 1, Exam 2, and Exam 3 then we can use the following

```
>>> grades.max()
```

```
100
```

If we wanted to find the maximum grade for each exam or equivalently apply the max function for each column then we can use the following

```
>>> np.max(grades, axis = 0)
```

```
array([ 98, 100,  96])
```

Now if we imagine that each row represents a student's grades for Exam 1, Exam 2, and Exam 3 and we wanted to find the maximum grade for each student's exam's then we can use the following

```
>>> arr.max(axis = 1)
```

```
array([ 96, 100,  98])
```

The same concept applies to other aggregate functions and scales to further dimensions. When handling arrays in higher dimensions, it is sometimes useful to supply a tuple or list as an axis parameter to specify where the function is applied.

Numpy Functions

np.round(array, decimals = 0) - returns the array, with all values rounded to a given number of decimals

np.sort(array, axis = -1) - returns the array, with all values sorted in ascending order, defaulting to the last axis

np.concatenate([array1, array2, ...], axis = None) - returns an array, joining a sequence of arrays on a given axis

```
>>> arr1 = np.arange(1,6)
```

```
>>> arr2 = np.arange(6, 11)
```

```
>>> np.concatenate([arr1, arr2])
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```

>>> arr1 = np.array([[1, 2],[5, 6]])
>>> arr2 = np.array([[3, 4], [7, 8]])
>>> np.concatenate([arr1, arr2], axis = 1)
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])
>>> np.concatenate([arr1, arr2], axis = 0)
array([[1, 2],
       [5, 6],
       [3, 4],
       [7, 8]])

```

np.where(condition, arr if true, arr if false) - returns an array in which the elements are replaced or unchanged based on a provided condition.

Consider the grades array used previously.

```

>>> grades = np.array([[90, 87, 96], [79, 100, 92], [98, 60, 74]])

```

If we want to find all grades that are greater than or equal to a 90 and replace the score with an “A” and leave all other values the same we can accomplish this using the following

```

>>> np.where(grades >= 90, "A", grades)
array(['A', '87', 'A'],
      ['79', 'A', 'A'],
      ['A', '60', '74'])

```

If we want to find all grades that are greater than or equal to a 90 and replace the score with an “A” and replace all other values with “Not A” we can accomplish this using the following

```

>>> np.where(grades >= 90, "A", "Not A")
array(['A', 'Not A', 'A'],
      ['Not A', 'A', 'A'],
      ['A', 'Not A', 'Not A'])

```

Now more realistically, what if we wanted to replace all grades with the letter grade equivalent. We can do this by using nested `np.where` functions.

```
>>> np.where(grades >= 90, "A",
             np.where(grades >= 80, "B",
                     np.where(grades >= 70, "C",
                             np.where(grades >= 60, "D", "F"))
                    )
            )
array([[ 'A', 'B', 'A'],
       [ 'C', 'A', 'A'],
       [ 'A', 'D', 'C']])
```

`np.loadtxt("fileName.extension", delimiter = " ", skiprows = 0, usecols = none, dtype = float)` - Returns an array of values read from a file (typically txt, csv, or tsv)

Imagine a file named `grades.csv`.

```
Exam 1,Exam 2,Exam 3
```

```
90,87,96
```

```
79,100,92
```

```
98,60,74
```

To convert this information into a numpy array, use `loadtxt`.

```
>>> grades = np.loadtxt("grades.csv", delimiter = ",", skiprows = 1,
dtype = "int32")

>>> grades
array([[90,  87,  96],
       [79, 100,  92],
       [98,  60,  74]])
```

```
np.savetxt("fileName.extension", array, delimiter = " ", newline =  
"\n", header = "", comments = "") - Writes an array to a file
```

Suppose we wanted to write grades to a tsv file. We can do this using the following

```
>>> np.savetxt("grades.tsv", arr, delimiter = "\t", newline = "\n",  
header = "Exam 1\tExam 2\tExam 3", comments = "")
```

Generating Random Numbers

Sometimes, you will want to generate random numbers. Here are a couple of ways to do so:

random.random() - returns a random floating number between 0 and 1

random.randint(start, stop) - returns a random integer between `start` and `stop` (inclusive)

Matrix Operations

Numpy is an incredibly powerful tool when it comes to optimization, machine learning, and AI. A big component of these algorithms is matrix calculations. As established above, Numpy data is stored in arrays which can be thought of as matrices when they contain integers or floats. Numpy is incredibly fast at performing any necessary matrix calculations because of its property of having all values be the same datatype.

For this section all sample code will be based on the following two arrays:

```
# Creating a 2x3 matrix  
matrix_a = np.array([[1, 2, 3], [4, 5, 6]])  
  
# Creating another 2x3 matrix  
matrix_b = np.array([[7, 8, 9], [10, 11, 12]])
```

Matrix Multiplication

While matrix addition and subtraction can be performed with normal numpy operations, performing matrix multiplication requires use of a specific function. This function is `np.dot` and performs the dot product of the two input arrays.

```
result_multiplication = np.dot(matrix_a, matrix_b.T)
```

****NOTE:** In order to perform matrix multiplication here `matrix_b` needed to be transposed using `.T` in order for the dimensions to align correctly

Matrix Inversion and the Determinant

Matrix Inversion can be performed using a built-in numpy linear algebra solver.

```
result_inverse = np.linalg.inv(matrix_a)
```

Additionally, we are able to find the determinant of a matrix using another function from `np.linalg`

```
determinant = np.linalg.det(matrix_a)
```

Eigenvalues and Eigenvectors

Numpy is also able to calculate the eigenvalues and eigenvectors for a given matrix.

```
eigenvalues, eigenvectors = np.linalg.eig(matrix_a)
```

This function returns a tuple of values which can be unpacked as seen above. The first value will always be the eigenvalues with the second value being the eigenvectors.

Resources

<https://numpy.org/>

<https://docs.scipy.org/doc/numpy/reference/generated/numpy.array.html#numpy.array>

https://www.tutorialspoint.com/numpy/numpy_data_types.htm

<https://towardsdatascience.com/navigating-the-hell-of-nans-in-python-71b12558895b>